parbench: Measuring How Coding-Agent Harnesses Handle k Concurrent, Conflicting Tasks

Team *vam* — Mid-Submission Progress Report (Phase 2)

Vishesh Saraswat
Aviral Malhotra
Manas Mittal
IIIT Hyderabad
India

Abstract

Parallel coding agents waste compute when tasks touch the same files: they re-explore the same context, overwrite each other and produce changes that no longer merge. No benchmark measures this beyond two tasks. We are building `parbench`, a pipeline that bundles k tasks from CooperBench into one repository snapshot, runs them under different orchestration strategies and harnesses, grades every task on the final tree, and logs tokens, time and conflicts. This report covers the architecture, the decisions made, the experiments run so far (Gemini and GPT-OSS through Antigravity, Qwen2.5-7B through OpenCode, a toy pool), the problems we hit, and the remaining work. Results are preliminary: so far we have evaluated small values of k (mostly k=2) with one repeat, and we plan a sweep over several k and seeds.

1 Problem and Motivation

Coding agents are now used on several tasks at once, by fanning out to subagents or by running one agent per task. When the tasks overlap, uncoordinated agents duplicate exploration, overwrite one another and produce patches that conflict at merge time. CooperBench [1] showed that two cooperating agents do worse than one agent doing both tasks, but it covers only pairs, one harness and a hand-built dataset. No benchmark lets us ask how orchestration strategy, harness and model affect *correctness, cost and wasted effort* as the number of concurrent tasks k grows. Our goal (Phase 1 proposal) is a benchmark and evaluation pipeline for exactly this, built on existing datasets and existing harnesses; we do not train a model or write a new agent framework.

Related work. CooperBench [1] evaluates two agents cooperating on one task with a hand-written dataset and a single harness. NP-Bench compares uncoordinated agents with up-front planning but contains only nine hand-built scenarios. Other studies report hidden costs of parallelism, such as workers reconciling inconsistent outputs, yet existing benchmarks either force sequential work, fix the number of peers, or cannot scale past two tasks with a controllable amount of conflict. Our additions are the k-task bundling, several harnesses and models on the same bundles, and cost and waste measured next to correctness.

2 System and Architecture

Vocabulary. A *task* is one CooperBench feature with hidden tests; a *pool* is the set of tasks sharing one repository at one base commit (CooperBench calls the pool a "task" and our task a "feature"); a *bundle* is k tasks from one pool in a fixed seeded order; a *run* is bundle × strategy × harness × model × repeat. CooperBench has 30 pools, 199 tasks and 652 pairs; we use its curated `core` subset (10 pairs, 9 repositories, four languages).

Table 1: The strategies. "Agent" means one harness session with one model.

ID	Name	What happens
S0	Isolated	Each task alone, one fresh agent per task; the k=1 reference used to measure how much is lost by running tasks together
S1	One agent	One agent receives all k tasks in one prompt and does them one after another; no subagents
S2	Parallel + merge	One agent per task, all in parallel, each in its own copy of the repository; patches are then merged (plain `git merge`, git's union driver, or an LLM resolver)
S3	Plan, then schedule	*Planned.* A planner predicts which files each task touches; tasks that share files run one after another, the rest in parallel
S4	Harness-native	One prompt with all k tasks to a harness with its own subagents enabled; the harness decides how to split and merge the work

Pipeline. (1) A bundle sampler builds k-task bundles. For $k > 2$ it extends each curated pair with more tasks from the same pool. (2) Each agent starts in a clean copy of the repository with the history after the base commit removed (CooperBench images contain the upstream future commits). (3) A strategy runs the agents. (4) If an agent ends by asking a question it is resumed with "No human is available; use your best judgement" (counted as a nudge). (5) We collect each agent's diff ourselves. (6) A grader applies the final tree, as one diff against the base, and runs each task's hidden tests with the pool image's own `runner.sh`; a task passes only if the runner exits 0 and at least one test passed. CooperBench's grader handles two features only, so ours handles any k. (7) A record per run stores correctness, tokens, wall-clock time, agent-busy time, conflicts, overlap and per-agent logs.

Strategies. A *strategy* says how the k tasks of a bundle are given to agents and how their work is combined (Table 1). S0–S2 and S4 are implemented and evaluated; S3 is planned. Prompt variants (plan first, other tasks visible) are in place.

Harnesses. Adapters exist for OpenCode, Pi, Claude Code, Codex CLI, Antigravity (`agy`) and Gemini CLI; each writes its own config, runs headless with JSON output, and a parser converts tokens to a common convention (input including cached tokens). Only OpenCode and `agy` have been checked against real output so far.

Sandbox. Agents run in Docker containers built from the CooperBench pool image, so they can run the project's own tests. A host

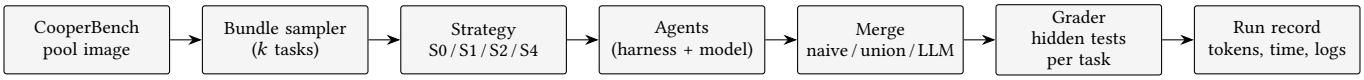


Figure 1: The `parbench` pipeline. Agents start from a clean copy of the pool image.

Table 2: Machines and software used for the reported runs.

	Development machine
Hardware	Apple M5, 10 cores, 24 GB unified memory (arm64)
Software	macOS, Docker Desktop (8 GB for containers); OpenCode 2.2.4 and Antigravity 1.5 in containers or on the host; Ollama for local smoke tests (Qwen3.5-9B, 64K context)
Used for	Antigravity runs (hosted Gemini 3.8 Flash and GPT-OSS 120B), pipeline validation, grading in Docker
	GPU server
Hardware	3× NVIDIA RTX 5000 Ada (32 GB each, shared; one GPU per run), Intel Xeon Gold 5317, 48 logical CPUs, 251 GB RAM
Software	Ubuntu 24.04, driver 560.35 / CUDA 12.6, vLLM 0.8.5 serving Qwen2.5-7B-Instruct (bf16, 32K context, Hermes tool parser), OpenCode 1.19.5 on the host (no Docker; prepared host pools)
Used for	OpenCode runs on the `core` Python pools; a Qwen3.8-27B (INT4) trial is planned

mode (temporary clone, no Docker) exists for `agy` and for OpenCode; a second no-Docker route was added using prepared host directories for use on a GPU machine.

3 Architectural Choices

- **Build on CooperBench** (pinned submodule, commit `63b9d44`) and re-bundle it, instead of writing tasks by hand.
- **Strategies first**: S0, S1, S2 and S4 (Table 1); the planner-based S3 comes later.
- **Docker sandbox per agent** and agents may run the repository's tests; hidden tests are never visible.
- **Fully autonomous agents**: no human in the loop, resumed with a fixed nudge, up to two times.
- **Compute**: local models for smoke tests; a GPU machine with vLLM for the main OpenCode runs; Antigravity for hosted Gemini and GPT-OSS.
- **Fairness**: same model for agents, resolver and planner; same step limit; one seeded task order per bundle.
- **Everything is logged raw** (JSON from each harness), so parsers can be fixed and runs re-scored without re-running agents.

4 Experimental Setup

Datasets. (a) A toy pool (6 features of a small Python package, known conflict structure) to check the pipeline. (b) CooperBench `core`: five Python/Go pools so far (dspy 8563, go_chi 27, click 2800, jinja 1559 and 1621), k=2, one repeat. **Grader check.** We validated the grader on 41 real tasks in six pools and on the six toy tasks. For each task, its hidden tests must fail on the unmodified code, pass with the reference (gold) patch, and pass with CooperBench's combined patch, which contains every task of the pool. All checks passed on arm64 images. **Hardware and software.** Two machines were used (Table 2). Runs on the two machines are not mixed in any comparison.

Table 3: `agy` on CooperBench `core` (5 pools, k=2, valid runs only, as of 6 Nov, 23:51 IST; the Gemini set is complete; GPT-OSS was stopped with some runs unfinished). n = valid runs (one run = one bundle); Task pass = fraction of tasks passing; Bundle = both tasks pass; wall and tokens are medians.

Model	Strat.	n	Task pass	Bundle	Wall (s)	In-tokens (M)
Gemini 3.8 Flash	S0	5	0.60	0.20	1350	16.0
Gemini 3.8 Flash	S1	5	0.70	0.40	554	6.8
Gemini 3.8 Flash	S2	5	0.70	0.40	774	14.2
Gemini 3.8 Flash	S4	5	0.70	0.40	620	5.6
GPT-OSS 120B	S0	5	0.20	0.00	1277	9.9
GPT-OSS 120B	S1	4	0.12	0.00	754	5.6
GPT-OSS 120B	S2	2	0.25	0.00	1320	12.8
GPT-OSS 120B	S4	2	0.00	0.00	188	1.0

Models and harnesses. `agy` with Gemini 3.8 Flash (low) and GPT-OSS 120B; OpenCode with Qwen3.5-9B (local, thinking off) on the toy pool and with Qwen2.5-7B-Instruct (vLLM; some planned runs had not finished) on the Python pools of `core`. **Metrics.** Task pass rate, bundle success, retention versus S0, pairwise conflict rate, overlap, tokens, wall-clock and agent-busy time, effective parallelism, nudges, and a validity flag. No dollar cost is computed for subscription models.

5 Results (Preliminary)

Toy pool, OpenCode + Qwen3.5-9B (local). All strategies pass all tasks: S0 k=2 (938 s), S1 (536 s), S2 (592 s, all three merge modes), S0 k=3 (1312 s). The bundle used had no conflicting pair, so the conflict path of the resolver was not exercised. These runs show the pipeline works end to end; they are not results.

Antigravity, `core`. Table 3 shows 33 valid runs. With Gemini, S1 (one agent, both tasks) passes 0.70 of the tasks against 0.60 for S0 (n=5 each). Because the isolated reference runs two agents, S1 uses 6.8M input tokens against 16.0M for S0 (2.4× fewer) and takes 554 s against 1350 s. S2 (parallel agents, then merge) passes 0.70 (n=5) at a pairwise conflict rate of 0.60 and an effective parallelism of 1.55; S4 (native subagents) passes 0.70 (n=5). The differences are not statistically meaningful with so few runs. GPT-OSS results are dominated by service failures (Section 6) and say little about the model. A single task costs about 3–8M input tokens, far above our initial estimate.

Per-pool view. Table 4 shows the pattern behind the averages. With Gemini, the jinja 1621 pair passes both tasks in S0 and S1, while the dspy, go_chi and click pairs pass exactly one of two in S0 and S1: one task per pair is consistently out of reach for this setup, whatever the strategy. We have not yet inspected which task fails or why. Because the agents could not run the repository tests on the host, some failures may be unverified mistakes, not missing capability.

Table 4: Task pass fraction per pool (valid runs; 1.0 = both tasks pass, – = no valid run yet).

	Gemini 3.8 Flash				GPT-OSS 120B			
Pool	S0	S1	S2	S4	S0	S1	S2	S4
dspy 8563	0.5	0.5	0.5	0.5	0.0	0.0	–	0.0
go_chi 27	0.5	0.5	1.0	0.5	0.5	–	–	–
click 2800	0.5	0.5	0.5	0.5	0.0	0.0	0.5	–
jinja 1559	0.5	1.0	1.0	1.0	0.5	0.0	0.0	0.0
jinja 1621	1.0	1.0	0.5	1.0	0.0	0.5	–	–

Table 5: OpenCode + Qwen2.5-7B-Instruct on the `core` Python pools (k=2, GPU machine; some planned runs had not finished). n = number of runs. Wall and tokens are medians per run.

Prompt	Strat.	n	Task pass	Wall (s)	In-tokens (K)	Conflict
default	S0	11	0.00	588	291	–
default	S1	11	0.00	98	47	–
default	S2	10	0.00	449	455	0.50
small-model	S0	7	0.00	875	974	–
small-model	S1	7	0.00	171	241	–
small-model	S2	7	0.00	451	720	0.43

OpenCode + Qwen2.5-7B (GPU machine, Python pools). Table 5 shows no task passing under S0, S1 or S2 (in all three merge modes), with the default prompt or with a "small model" prompt. The model did attempt the tasks: only a few agent runs ended with an empty patch. According to our log analysis (in the repository's `docs/` folder), failures are mostly broken code (syntax and import errors, truncated edits), not wrong-but-valid implementations, and merging does not rescue them (merge conflict rate about 0.4). The small-model prompt removes the model's wandering (git and scratch-file commands drop to about zero) and multiplies real edits, but does not change the pass rate and increases time and tokens. S1 reaches genuine assertion failures more often than S0 or S2, a hypothesis we have not tested. This model therefore cannot rank the strategies on success, only on time, tokens and conflicts; a stronger model (Qwen3.8-27B, INT4, is planned) is needed. Some bundles appear twice in reversed task order and are extra samples, not independent bundles.

6 Problems Faced

Making CooperBench usable for $k > 2$ took real work: its grader and runner handle only two features, and its images leak future upstream history, so we wrote our own grader and a history scrub. Each agent harness behaved differently from its documentation, which required adapters verified against live output, and keeping results trustworthy cost us several reruns: agents found and searched for the upstream fix on the web, and service quota and capacity errors silently ended runs early. We now block web tools, retry through service errors, mark affected runs invalid, and audit the logs. The full benchmark is about 3,900 agent sessions for k=2 alone, so we evaluate curated subsets.

7 Limitations

We have few runs per configuration (at most five bundles, one repeat each), so confidence intervals are wide. `agy` runs on the host, so agents cannot run the repository's tests, which lowers absolute pass rates and makes them not directly comparable with containerised OpenCode runs; subagents cannot be disabled in `agy` S0–S2 apart from our custom agent. Model quotas and server capacity are outside our control. Only k=2 has been run, and only one harness has results on real pools with a model strong enough to pass tasks.

8 Milestones and Changes to the Proposal

Achieved so far. An end-to-end pipeline from bundle sampling to graded, logged runs; a k-task grader checked on 41 real CooperBench tasks and a toy pool; four strategies (S0, S1, S2 with three merge modes, S4) and prompt variants; six harness adapters, two verified on real output; a Docker sandbox and two no-Docker routes; first measurements with three harness/model setups; a proof of concept for SWE-smith bundles; and tooling the proposal did not foresee: a web-use audit, a run-validity flag, automatic retry through quota errors, and raw-log redaction.

Changes. (1) *Models*: the course encourages local models, so the main OpenCode runs use open models served with vLLM, while hosted Gemini and GPT-OSS through Antigravity act as a stronger reference. (2) *Scale*: the full benchmark is not feasible (Section 6), so we evaluate the curated `core` subset first, with k=2, and move to k up to 6 afterwards. (3) *Optional items*: bundling SWE-smith bugs to add an adjustable conflict-density dial remains "if time permits". (4) *Risk*: access to hosted models (quota and capacity) turned out to be a schedule risk; we plan the final experiments around local models.

9 Planned Task Generation with SWE-smith

SWE-smith [2] is an MIT-licensed toolkit that builds an execution environment for a repository and then synthesises task instances in it. Each instance is a bug patch that breaks previously passing tests and carries its FAIL_TO_PASS and PASS_TO_PASS test lists, a generated problem statement and the repository's shared Docker image. The authors report about 50k instances from 128 Python repositories, created by procedural code edits, LM modification or rewriting of functions, reverted pull requests, and combinations of validated bugs from the same file or module. Because all instances of one repository are patches on a single pinned commit, many tasks can be placed on one snapshot. This is what our benchmark needs and what CooperBench's 30 pools cannot provide at scale; it is also the "adjustable conflict density" option named in our proposal.

Plan. (1) Take validated single-function bugs from repositories SWE-smith supports, preferring ones CooperBench also uses (jinja, click, dspy) so the two sources can be compared. (2) Index each bug's file, function and line range by parsing its patch, since instances do not store them separately. (3) Group k bugs by overlap: different files, the same file in different functions, or the same function; the share of overlapping task pairs becomes the conflict-density dial. (4) Apply all k patches to the clean commit, confirm that each task's failing tests still fail, and reject bundles whose patches do not apply or whose tests interfere. (5) Run every harness and strategy on the bundle and grade each task on its own FAIL_TO_PASS tests on the

Table 6: SWE-smith proof of concept (oauthlib, k=4). Same fn: share of task pairs in one function; Near: pairs within 3 lines; Conflict: share of task pairs whose fixes conflict under `git merge`; Restored: bundles whose merged fixes restore the clean code.

Overlap level	Bundles	Same fn	Near	Conflict	Restored
L0 different files	12	0.00	0.00	0.00	1.00
L1 same file, different functions	12	0.00	0.00	0.00	1.00
L2a two bugs in one function	12	0.17	0.00	0.00	1.00
L2b all k bugs in one function	12	1.00	0.01	0.00	1.00
L3 two bugs on adjacent lines	10	0.17	0.17	0.00	1.00

final tree with our existing grader. Step-by-step details with sources are in `docs/swesmith_guide.md`.

Proof of concept. We ran steps (1)–(4) on the oauthlib instances of the SWE-smith dataset (one repository, one pinned commit): 558 of the fetched instances apply to the repository mirror, and 308 of them touch exactly one file and one function in git's minimal diff. We built 4-task bundles at five overlap levels (10–12 each), applied all four bug patches to the clean commit, and, for every task, built the patch that undoes only that bug, which is what a correct agent fix amounts to. Table 6 shows that the dial controls overlap exactly (the share of task pairs in the same function rises from 0 to 1.00), that every bundle is valid (its patches apply together), and that merging all per-task fixes restores the original code in every case. It also shows a negative result: **no textual merge conflict arose at any level**, including ten bundles with two bugs on touching lines. The dial therefore does not yet give conflicting tasks. SWE-smith bugs rarely change the same lines, and two bugs on the same line cannot be placed in one snapshot, so we expect conflicts to come mostly from semantic interaction between agents' fixes. Testing that needs agent runs with SWE-smith's test environments, which we have not done, and we have not yet explained why the touching pairs merged cleanly. The script is `scripts/swesmith_poc.py`.

Open questions. SWE-smith's failing tests are visible to the agent, so we must decide whether to hide them at solve time; its combine step works only at file or module level, so same-function bundles need independently generated bugs; and we have not yet verified that its pinned commits match our pools, or that bundled bugs keep their individual test sets.

10 Remaining Work (to 30 November)

- Finish the remaining GPT-OSS runs and the heavy `core` pools (outlines, typst, tiktoken, llama_index, react-hook-form), and add repeats (≥3) with confidence intervals; the Gemini set is complete for the five pools run so far.
- The main OpenCode experiments on the GPU machine with a stronger open model (Qwen3.8-27B, INT4, then a 35B-class model), since Qwen2.5-7B passes nothing.
- $k = 3, 4, 6$ bundles and an analysis by conflict density, using CooperBench's gold conflict report now and SWE-smith bundles with a controlled overlap next.
- Validate the Pi, Claude Code and Codex adapters on real output; cache S0 results across bundles.
- Our own contribution: a predict-then-schedule strategy (S3) that predicts each task's files, runs conflicting tasks one after another and the rest in parallel, evaluated with file-prediction recall and the cost saved.
- Run agents on SWE-smith bundles with their test environments to see whether semantic conflicts appear where textual ones do not, and complete the GPT-OSS runs.
- Report cost and resource use (tokens, GPU-hours) as in the proposal, and the final ACM-format report.

Table 7: Key files we reviewed (all under `src/parbench/` unless noted).

File	What was checked
`grader.py`	pass criterion, agent tests removed, clean reset
`strategies.py`, `merge.py`	S0/S1/S2/S4 logic; merge modes, conflict rate
`agent_runner.py`, `runner.py`	nudges, retries, token and metric computation, validity flag
`sandbox.py`, `dataset.py`	history scrub, host mode, bundling, subsets
`harnesses/cli_agents.py`	per-harness commands and token conventions
`prompts/*.md`, `configs/agy_agents/`	exact agent prompts; disabled tools
`configs/experiments/*.yaml`, `models.yaml`	settings behind every reported number
`docs/findings.md`, `results/*/run.json`	claims and table values against raw logs

11 Use of AI Tools

Tools. Most of the pipeline code, prompts, documentation and the first draft of this report were written with Claude Code (Claude Opus 5.5 and Sonnet 5.5), including research subagents that read the CooperBench repository and the agent CLIs. AI agents are also the *subjects* of the experiments (Gemini, GPT-OSS, Qwen through OpenCode and `agy`); their outputs are data, not part of our code.

The decisions are ours. The AI proposed options and implemented them; we made the decisions on models, harnesses, strategies, tasks (the `core` subset, k=2 first), compute, the sandbox design, and the removal of web tools to prevent contamination.

Verification. Outputs were checked by running the grader on gold patches (41 real tasks and 6 toy tasks), dry-running every experiment plan, probing each harness against the real tool, scanning logs for secrets and web use, and correcting the AI's first attempts where they were wrong (documented in `docs/findings.md`). In addition, we read and checked the key files in Table 7. The prompts, a summary of the AI's outputs and the checklist are in `docs/ai_usage_log.md`.

Repository

https://github.com/vishesh312/autose-project

References

[1] CooperBench: benchmarking coding-agent cooperation. arXiv:2601.13295, 2026 (code: github.com/cooperbench/CooperBench).

[2] J. Yang et al. SWE-smith: Scaling data for software engineering agents. arXiv:2504.21798, 2025 (NeurIPS 2025 Datasets and Benchmarks; code: github.com/SWE-bench/SWE-smith).